

Node and Tuple Embedding

In this assignment, we will focus on embedding methods for database tuples.

As the database we continue to use [rel-stack](#), from the relbench dataset suite, and as the task we continue with [user-engagement](#) (predict for each user if they will make any votes, posts, or comments in the next 3 months).

In the previous assignment, you produced a feature matrix by extracting relational features (using joins and aggregations) and structural graph features that you added to the matrix. In this assignment we will use this feature matrix as a basis for learning different embeddings.

Submission Guidelines

Submission is in **pairs** only. Your submission should include a zip of the following:

1. A **report** (pdf file) containing your results and the answers to the questions in this assignment (in order). Note that the last two questions are **summary questions**; it may be useful to populate your response to them throughout your work.
2. Package usage: you may use any existing python packages you find helpful. Detail them in your answer to question 14.
3. You may reuse any code you have written in the previous assignment.
4. LLM/AI usage: you may use AI tools as much as you like. **You will need to report your usage of LLMs in detail** (see last question in this assignment).
5. All of the code you used, written in python, and specifically, a jupyter notebook containing all the results included in your report (graphs, training measurements, etc.) and the code used to generate them. You may use .py files imported into the notebook, for convenience and readability of your code. Export the notebook (including the results of running it) to html format using the following command:

```
jupyter nbconvert --to html your_notebook_with_results.ipynb
```

Part 1: Train Base Embeddings

For all the embeddings that you train in this exercise, use a dimension size of $k=128$.

Tuple Embedding via Dimensionality Reduction

1. Remove the label column (`contribution`) from the feature matrix. To get a numeric matrix: (1) Remove textual attributes. (2) Impute missing values using `sklearn.impute.SimpleImputer(strategy='median')`. **Remove attributes where all values are missing.** (3) For timestamp attributes, transform them to the difference from the target timestamp. For example: consider the tuple:

timestamp	Id	CreationDate	contribution
21/12/2010 00:00	123	11/12/2010 12:00	1

Transform `CreationDate` to `DaysSinceCreation`: 9.5 (since the creation date is 9.5 days before the timestamp.)

On the resulting numeric matrix, compute an embedding of size k using kernel PCA (idea: truncated SVD over the feature matrix, using rbf kernel for the proximity).

Handling the large size of the matrix: creating the kernel matrix will not fit in memory. To approximate Kernel PCA on a large matrix, use

`sklearn.kernel_approximation.Nystroem` to project the data into an approximate RBF kernel space, followed by `sklearn.decomposition.PCA` to reduce the dimensions to k .

Node Embeddings

2. As you have done in the previous exercise, construct a graph representing the database, where each tuple is represented by a node, and the edges are defined by PK-FK connections. This time, **instead of using `networkx`**, use the built-in functions in `RelBench` to get the dataset as a heterograph and transform it to homogenous using `to_homogeneous()`. Then use `pytorch-geometric` (or other) for the embedding algorithms.

Tuples to include: For simplicity, use all tuples in the database with timestamp **up to and including** `dataset.val_timestamp`.

Attributes to include: Remove all attributes except for PKs and FKs, since tuple features are not used in the graph embedding methods used in this exercise.

After building the graph, report the following graph statistics:

- a. number of nodes
 - b. number of edges
 - c. Min, max, avg and median of node degrees.
3. On the resulting graph, train embeddings using the following methods.

- a. Laplacian eigenmaps, using the adjacency matrix as proximity. (For that you would need to transform the graph to undirected.)
Handling the large size of the matrix: Note that `sklearn`'s spectral embedding will not be able to handle the large matrix size; instead, use `randomized_svd`, from `sklearn.utils.extmath`, which implements a randomized approximation. Make sure to represent the matrix sparsely as a `csr_matrix`.
- b. Node2Vec ([implementation](#)) with a walk length of 24, a context size of 5, and the adam optimizer with learning rate 0.01. For data loading, use a batch size of 128 and shuffle the samples. **If the batch size is too large for the GPU available memory, try decreasing it.** Train for 25 epochs.
- c. Metapath2Vec ([implementation](#), [paper](#)) - for this you would need the heterograph (before transforming it to homogenous). Define the allowed edges in the metapath according to the database schema, but make the graph undirected so that walkers don't get stuck. Use the same parameters as for Node2Vec. For the metapath, use: *user-post-comment-user*.

Note: For (b) Node2Vec and (c) Metapath2Vec, use the `pyg` implementation. PyG is a package to handle machine learning over graphs.

Part 2: Analysis

4. **Visual analysis:** Sample 200 tuples with `contribution=1` and 200 tuples with `contribution=0` from the validation table. For each of the 4 embedding method, use t-sne to project the embeddings of the sampled tuples to 2 dimensions. Color the scatter graph in 4 different ways:
 - a. The binary label in the prediction task (`contribution`) – positive and negative tuples have different colors.
 - b. The degree of the node (0, 1-10, 11-100, >100)..
 - c. The number of posts each user has - above 2 posts, below 2 posts.
 - d. Badges - does the user have at least 1 badge.

Add all 16 figures to your report with matching captions. Arrange them in a 4x4 structure for convenient comparison (one row for each embedding method, column for each coloring). Describe any structure or pattern that you see emerging from these visualizations.
5. **Analysis via downstream performance:** for each embedding method, use the resulting embeddings as input to a classifier aiming to predict user engagement. Using the model that performed best in the first homework assignment, train the classifier on the embeddings and report the performance: accuracy, AUC, precision, recall, F1 over train and validation. Report the results in a table of the following structure:

	Accuracy train/val	ROC AUC train/val	Precision train/val	Recall Train/val	F1 train/val
Kernel PCA					
Laplacian eigenmaps					
Node2Vec					
Metapath2Vec					

6. Repeat the classification process for combinations of embedding methods: for each node, create a combined embedding by concatenating its embeddings in different methods. **Add the performance metrics to the table** from the previous question. Use the following combinations:
 - a. Kernel PCA $\oplus$ Node2Vec
 - b. Kernel PCA $\oplus$ Laplacian Eigenmaps
 - c. Kernel PCA $\oplus$ Metapath2Vec
 - d. Node2Vec $\oplus$ Metapath2Vec
 - e. Emb $\oplus$ Emb (For the best performing embedding method “Emb” from the previous question). The goal of this variation is to serve as a baseline, since concatenating the embeddings increases the number of learned parameters in the classifier.
7. Based on your answers to questions 4-6, answer for each embedding method (including the concatenated embeddings): how well did the embedding capture differences between the two populations? Discuss the results: which embedding methods led to the best predictive performance?
8. Compare the performance results to those you got in assignment 1 by generating tabular and graph features. Did any embedding method yield improved performance over using generated features?
9. **Qualitative analysis:** sample or choose a node u (representing a user) from the graph, and create a visualization of their 2-hop neighborhood. For each visualized node v , add to the figure (possibly next to graph, if you find it more convenient): their type in the graph (i.e., the table it came from), its ID, and the embedding similarity of the node v with u , in each of the base embeddings (from the table in q5). Repeat this process for **3 different nodes u** . Describe any differences you find between the embedding methods in these visualizations.

Part 3: questions

10. For each base embedding method you used in this assignment (Kernel PCA, Node2Vec, Laplacian Eigenmaps, Metapath2Vec), explain how it instantiates the encoder-decoder

paradigm (What are the decoder, loss, and proximity definitions)? As a result of the choice of these elements, what does each embedding method aim to preserve?

11. Why are embedding methods usually considered as task-agnostic?
12. In questions 1 and 3a you have used two different methods to handle the large size of the matrix. Explain, in high level, how each of them works. Could we have used the `random_svd` method for the `kernelPCA` in question 1, and why?
13. In `metapath2vec`, what are the constraints on the length of the random walk compared to the length of the metapath? (Should one be longer than the other? Are there any other constraints?) And why?
14. **Summary** question: write a conclusion for your report. In your answer:
 - a. Describe the steps you took and what you learned in each of them.
 - b. Describe the differences between the tabular feature extraction process (from the previous assignment) and the process of learning embeddings. What decisions does a data analyst have to make in each of them? What are the advantages and disadvantages of embeddings compared to feature engineering?
 - c. Based on your results, what were the advantages and disadvantages of using embedding trained on the graph, compared to tabular embeddings, and vice-versa?
15. **Packages you used:** if you used any packages other than those mentioned in this assignment specification, list them in your report, and describe what you used them for (about a sentence per package).
16. **AI usage:** you are more than welcome to use AI in your work. Add a paragraph to your report, describing in detail **where** you used AI tools/LLMs (such as `ChapGPT`, `Gemini`, etc.). Explain your **critical validation process**—how did you catch hallucinations or optimize the AI's suggestions for the specific tasks? Additionally, estimate the **percentage** of the total workflow in the assignment that was assisted by AI tools. **You can use AI as much as you want, as long as you understand what it did, validate that it was correct, and report it.**